Writing programs that write programs is what metaprogramming is all about. In this chapter you saw how literate CoffeeScript works and how it can change the way you think about program source. You then saw how to adapt existing CoffeeScript syntax and create domain-specific languages that provide a closer fit than plain CoffeeScript for certain types of problems. Finally, you saw how to modify CoffeeScript itself from inside a CoffeeScript program.

While learning what works in CoffeeScript, you also saw that CoffeeScript isn't a Lisp. The CoffeeScript compiler is written in CoffeeScript, but it doesn't have a meta-circular evaluator, and the `eval` function is mostly a distraction. Another way that CoffeeScript is not a Lisp is that it doesn't have macros. That said, there are moves to implement hygienic macros in JavaScript. If you're interested in that, then you should check out the sweet.js project (sweetjs.org). It's also possible to create macros for CoffeeScript, but that's a lesson for another day.

In the next chapter you'll look at how the existing CoffeeScript syntax and semantics work in asynchronous environments.